GAME DESIGN DOCUMENT

Star Bazaar

A single-player space trading game



VERSION	DATE	ENGINE	SOURCE
Forth Draft	21 October 2026	Godot 4 · macOS	github.com/Veltrynox/star-bazaar

Contents

— Revision Notes & ELVTR Review 03

01 Executive Summary / Game Specificity 04

02 Game Mechanics / Player Experience 05

03 AI Architecture 10

04 Constraints 13

05 Engine Integration 15

Draft 3 → 4
Changes From the Third Version

Win rule	The win is now the first time cash reaches the target at the end of any cycle. Loss is still checked at the end of cycle 30.
Win threshold	Lowered from 12,000 to 9,000.
Travel and cycle time	One cycle is 2 minutes, and every leg takes 1 cycle.
Trade example	The “about 190” sentence is now 140 per round trip, with maintenance charged per cycle.
Tutorial window	“First minute” is now “First 4 cycles (about 8 minutes)”.
Pace markers	Replaced by a goal line showing the target and current cash.
Loan	Removed.
Ship upgrades	Still present, now labeled part of the core loop.
Permits	Permit margin bonus is 50%. System 3 can only be bought after System 2.
Debug menus	The removal from release builds is stated.
Agent crew	Rebuilt as six agents modelled on a small indie studio. UI Designer and Platform Engineer are merged into the Gameplay Programmer, and the Budget Controller into the Producer. Weekly limits total 68%.

— ELVTR REVIEW

> **The agent stress-test** — What did the crew flag?
> The crew mostly flagged balance, economy and goal issues. It helped me to fine-tune the game so it’s better structured number-wise. Some agents flagged the debug menu being in the game as an exploit, but for me it was obvious it will be stripped from the final version.

> **Feedback from peers or your own review** — What did humans catch?
> I didn’t have a chance to show it to peers, but in my own judgement the game feels a bit boring and fast so far, uneventful maybe. But I hope that after the debug menu and core loop are there I can balance it and add more content. I would like to start really small with a super small prototype first that works and looks good, and build from there.

01

Executive Summary / Game Specificity

ONELINER	Star Bazaar is a single-player space trading game
NAME	Star Bazaar
GENRE	Space trading game
ENGINE	Godot 4 (GDScript)
PLATFORMS	macOS
CORE LOOP	Check prices, buy where a commodity is cheap, fly, sell where it is in demand, and reinvest in ships and permits. Each new ship runs the same loop as an automated route.
WIN	The first time the cash balance is at least 9,000 credits at the end of any cycle, the game ends and the success story shows.
LOSS	If the cash balance is still below 9,000 credits at the end of cycle 30, the game ends and the end screen shows the cash reached and the target.

02
Game Mechanics / Player Experience

The numbers come from a rough paper model built on the GDD's fixed facts: `win_cycle` cycles of about `cycle_minutes` minutes each. They're meant to give a working first pass for tuning in the debug menu, not final balance.

The game takes about an hour to complete. In game cycle lasts around `cycle_minutes` minutes. There are `win_cycle` cycles.

The player can pause the gave at any time.

— CREDITS INFLOW

- Trades
- Events

— CREDITS OUTFLOW

SHIP MAINTENANCE	`maintenance` per ship, every `maintenance_interval` cycles
TRIP COST	Every trade trip cost (we combine fuel price, crew price here just to keep less variables). `trip_cost`cr per leg.
PERMITS	One time purchase to open a new solar system.

- Events

— MECHANICS

TRADES	Trade buy and sell resources
PERMITS	Solar systems access
EVENTS	To bring some randomness and fun to the game.

— TOP BAR

GOAL LINE	"Goal: `win_cash` by cycle `win_cycle`. Now: [cash balance]."

The game is mostly event and text driven, for now.

FIRST 4 CYCLES (ABOUT 8 MINUTES) OF THE GAME

The player reads the preface story. The game starts. We sees 3 Planets, a notification board on the right side with a mission hint.

"Buy a ship" mission text. The UI on top with some information: `starting_credits` starting credits. The planet highlights, the players clicks on it, a menu pop ups. This planet has an abundance of evergy abundance but not enough food. Under it there is a menu with a ship we can buy. The Player clicks on it and buys a ship. Mission complete. We see another menu that explains how to trade. The Player clicks on another Planet and sees that that the planet has abundnce of food but low energy supply. The Player buys energy cheaply at `energy_buy` per unit, on one planet and clicks on the second Planet and sells the energy for `energy_sell`cr. But the player can't simply sell and fly back, it would be a low cash flow that ship maitanance and trip cost will eat soon. The Player buys food cheaply for `food_buy`cr and flies back to sell it on Planet 1 where food sells for `food_sell`cr. The automated menu automatically asks if we want to establish an automated trade route. We hit yes.

That's a profit of 8 per unit. With 20 units of cargo, 160 per leg, 320 per round trip. A round trip takes 2 cycles, so it costs 80 trip cost and 100 maintenance (50 per cycle), leaving about 140 per round trip.

After several cycles we have enough money to get a second ship. The Player can choose if it adds the ship to the existing route or create a new one. Too many ships eat the resources on the planet and increase the price of the resource. Too few give low cache flow. There is a cap of possible ships amount which is `ship_cap` ships max.

AUTOMATED ROUTES

– `saturation` sell price per extra ship on the same route

That's the code loop pretty much. I imagine the game to be a slow pased and builds up interest over time with more updates. I dont worry that the first prototype is just that. The working core loop.

BALANCE TARGETS

A decent player buys their 4th ship around cycle `target_4th_ship_cycle`, gets the first permit around cycle `target_first_permit_cycle`, and has `target_ships_range` ships by about cycle `target_8_ships_cycle`. That player's cash balance reaches `win_cash` around cycles `target_win_cycle_range`.

A sloppy player stays on one route, overloads it, or skips permits. That player ends somewhere around `target_sloppy_cash_low`–`target_sloppy_cash_high` and loses. The loss condition matters without being punishing.

Selling and flying back empty still earns, but only half as much. One leg earns 160 and costs `trip_cost` to fly, so the ship makes 120 before maintenance. Carrying cargo both ways makes 240 per round trip before maintenance.

If players win before about cycle `check_early_win_cycle`, raise the win threshold or the maintenance cost first. f the second ship comes later than about cycle `check_second_ship_cycle`, the tutorial pacing drags. Raise the margins slightly.

Expose all of these values in the debug menu. The QA bot runs can then check whether “always expand” and “never expand” strategies both lose. If one of them wins, it’s a dominant strategy.

EVENTS

Events can change the resource amount on the planet by `event_effect` and open and close some Planets temporarily. Events happen every `event_interval` cycles (about 7 events per game, roughly one every 8 minutes)

MENU UI

DEBUG MENU	"Debug and Game Control menus are removed from the release build"

EVENT CONTROL

- How often events happen
- A list of events availaible
- Trigger an event
- Effect multiplier for events

GAME CONTROL MENU

Balance values (v0.1)

Change here to test. Each value is defined once in this section; the rest of the GDD refers to it as `key`.

`cycle_minutes`	2 minutes
`starting_credits`	1,500
`ship_price`	1,000
`cargo`	20 units
`ship_cap`	8 ships
`energy_buy`	10
`energy_sell`	18
`food_buy`	12
`food_sell`	20
`trip_cost`	40 per leg
`maintenance`	150 per ship
`maintenance_interval`	3 cycles
`saturation`	10%
`permit_2_cost`	2,000
`permit_3_cost`	4,000
`permit_margin_bonus`	50%
`event_interval`	4 cycles

event_effect	±10–25% or ±300–800 cr
win_cycle	30
win_cash	9,000
pace_cycle_10_target	4,000
pace_cycle_20_target	8,000
pace_cycle_25_target	10,000
upgrade_cost	1,000
upgrade_margin_bonus	10%

Balance targets (checks)

target_4th_ship_cycle	9
target_first_permit_cycle	13
target_8_ships_cycle	18
target_win_cycle_range	25–28
permit_1_affordable_cycles	12–14
check_early_win_cycle	22
check_second_ship_cycle	6
target_ships_range	7–8
target_sloppy_cash_low	7,000
target_sloppy_cash_high	10,000
One cycle lasts:	2 minutes
Travel time:	Every leg takes 1 cycle

A Player makes money mostly from trades.

Planned features after we create the working core loop game.

— SHIP UPGRADES (PART OF THE CORE LOOP, THE LATE-GAME SPENDING OPTION)

Once a ship is bought, the player can pay `upgrade_cost` credits to upgrade it. The upgrade adds +`upgrade_margin_bonus` margin per unit on that ship's route. Each ship can be upgraded once. Upgrades are optional.

PLANNED LATER	Space buildings trade posts.

THE WIN CONDITION

Told in the beginnig of the game over a text menu. The Player has a dream to escape from society on its own planet. But in order to do that, the player needs money. The first time the cash balance (the credit figure in the top UI bar) is at least 9,000 credits at the end of a cycle, the game ends and the success story shows. If cash is still below 9,000 at the end of cycle 30 (about 1 hour in), the game ends with a loss.

SHIP STATS

SHIP PRICE	`ship_price` cr
CARGO PER SHIP	`cargo` units

TRADE

About 8 credits margin per unit. A round trip earns a margin of about 320 per ship (sales minus buying cost), before trip cost and maintenance.

PERMITS

The first permit becomes affordable around cycle `permit_1_affordable_cycles`. New systems have about `permit_margin_bonus` better margins than the starter system, to reward expansion.

SYSTEM 2	`permit_2_cost` cr. System 3: `permit_3_cost` cr

System 3 can only be bought after System 2.

03
AI Architecture

I (me the owner) works in a claude code session. Claude leads and writes the main code. and subagents take bounded pieces of work in parallel.

The only game is be playing the build. Every step ends with something I can play and a short list of questions for me.

— THE CREW: 6 AGENTS, MODELLED ON A SMALL INDIE STUDIO

The reference is a small indie team of five or six people, as on compact strategy and trading games. In a team that size, everyone owns one discipline and nobody exists only to manage the others. The crew below works the same way. Weekly limits add up to 68%, which stays under the GDD's 70% cap.

01 Creative Director

CLAUDE SONNET

ROLE	Owns the vision and the GDD. Approves or rejects every other agent's output against it.
INPUT	The GDD sections and each agent's JSON handoff.
OUTPUT	A keep / cut / change verdict with a one-line reason per item, plus GDD updates.
EFFECT	Keeps scope small and makes sure each build stays true to the "escape to your own planet" fantasy.

A studio needs one person with final say on design, or features drift.

WEEKLY LIMIT **5%**

02 Producer

CLAUDE HAIKU

ROLE	Turns the build plan into an ordered task queue. It also tracks the usage meter and pauses work when a window passes about 80%.
INPUT	The GDD build steps, agent handoffs and current meter usage.
OUTPUT	A `schedule.json` file with the task, owner, dependencies and status for each item.
EFFECT	Work happens in the right order and the budget never runs out mid-task.

Real studios have a producer, not a separate budget person. The budget controller is folded in here because tracking spend is part of scheduling.

WEEKLY LIMIT **2%**

03 Systems & Economy Designer

CLAUDE SONNET

ROLE	Owns every number in the game: prices, margins, maintenance, saturation, permits and event effects. It runs a headless paper simulation to test them.
INPUT	The balance table, planet data and the QA reports.
OUTPUT	A versioned `balance.json` file and a short note on what changed and why.
EFFECT	Trading stays profitable but never trivial, and the win lands around cycles 25–28.

The whole game is an economy. If the numbers are wrong, no amount of code or art saves it.

WEEKLY LIMIT **6%**

04 Gameplay Programmer

CLAUDE SONNET

ROLE	Writes all game code in GDScript: the core loop, routes, events, UI panels, the debug menu and the macOS export preset.
INPUT	The task brief, the relevant GDD sections, `balance.json` and the Godot project files.
OUTPUT	Complete Godot scripts and scenes, plus a playable build at the end of each step.
EFFECT	Creates the backbone of the game, so every step ends with something the user can play.

Someone has to build the game. This agent also absorbs UI and platform work, because in Godot both are just scenes and project settings.

WEEKLY LIMIT **40%**

05 Technical Artist – Galaxy Builder ESSENTIAL

CLAUDE SONNET

ROLE	Builds the galaxy background. It scatters a dense point cloud, bakes a low-res 3D density grid, picks the sun points and propagates their light through the volume once at scene generation.
INPUT	The Godot project, the target look (map screenshot) and a performance budget for macOS.
OUTPUT	A generator script, baked point and grid data, and the shader or material setup.
EFFECT	Gives the game its signature look, where distant and hidden stars fall into darkness. The map reads as space, not a spreadsheet.

This is specialised tech-art work, the closest match to the user's own VFX background. It also has no overlap with gameplay code, so it can run in parallel.

WEEKLY LIMIT **10%**

06 QA Analyst

CLAUDE HAIKU

ROLE	Runs scripted bot playthroughs using several strategies: always expand, never expand, single route and optimal. It flags crashes, softlocks and dominant strategies.
INPUT	The latest build, the bot scripts and the previous report.
OUTPUT	A `bug_report.json` file with run_id, strategy, final cash, cycle reached, issue_type and severity.
EFFECT	Broken or unwinnable runs never reach a human playtester, and balance claims are checked against real runs.

The agents that build and balance the game shouldn't grade their own work. Studios keep QA separate for exactly that reason.

WEEKLY LIMIT **5%**

The trade-off: the Programmer now carries the UI and platform work, so it's the bottleneck. That's acceptable for a prototype with a few panels and one platform. It's also the first place to add an agent back if the UI grows.

04
Constraints

Agents spend at most 70% of the weekly meter. The other 30% is reserved and never assigned to an agent.

Sonnet costs ~20x Haiku per token, so its 1.8M tokens are ~99% of the weekly cost in Section 3.

The Haiku agents together are small on the meter.

HARD STOPS (WRITTEN INTO EVERY AGENT PROMPT)

> Stop when the task is done, not when time runs out.
>
> Stop at the turn cap even if unfinished; write a handoff
>
> Never re-read a file already read this session.
>
> No exploratory work outside the task brief.
>
> If a task looks bigger than its cap, stop and report the estimate to the Producer instead of starting.

Every agent ends its session with this envelope, in a JSON file:

`agent:`	<agent name>
`date:`	<YYYY-MM-DD>
`task:`	<one line from the Producer's queue>
`status:`	done | partial | blocked
`files_changed:`	<list of paths>
`schema_version:`	<version of any JSON written>
`usage_note:`	<rough % of meter used this session>
`open_questions:`	<max 3 bullets, or "none">
`handoff:`	<max 150 words: what the next session needs>

RULES

> Data goes in JSON that matches the versioned schema
>
> Prose summaries: max 200 words. No restating the GDD.
>
> No invented fields, planets or values. Unknown → write "UNKNOWN" and list it under open_questions.
>
> Code (Programmer only): full files, no partial snippets, one file per change block.
>
> Lead Designer reviews: keep / cut / change + one-line reason per item. Nothing else.

Pro limits come in 5-hour session windows plus the weekly meter, not requests per minute. Plan around windows:

> The Gameplay Programmer gets its own window. Nothing else runs in that window.
>
> Haiku agents are batched: up to 4 per window, run in dependency order (Economy → Producer → UI → QA).
>
> Stop starting new agent work when a window passes ~80%; finish the current task and write the handoff.
>
> Max 2 agent windows per day, leaving room for my own use.
>
> If a window limit is hit mid-task, the agent's handoff is the restart point. No re-running from scratch.

RULES

> Start every session fresh with a brief: max 300 words plus the files listed above.
>
> Programmer: start a new session when the task changes or the conversation gets long; carry over only the handoff.
>
> GDD is sent in sections, never whole, except for the Lead Designer's Section 1-2.
>
> Custom instructions per agent: max 150 words, kept in one file (agent_rules.md) so they are written once, not retyped.

Also a constraint is my own available time outside the course, which caps how many build iterations a week can realistically absorb.

05

Engine Integration

I'll use MCP and an agent that writes code directly to the game engine. That agent collects the output json file and makes necessary fixes.

Every agent's output is translated into a fixed, versioned JSON schema, and only that JSON crosses into the engine. Gameplay programmer translates it to GODOT.

GALAXY GENERATION	Scatter a dense point cloud, bake a low-res 3D density grid from it, pick the sun points, and propagate light from the suns through the volume, dimmed by density. This runs once at scene generation, so distant and occluded stars darken.